DLCD LEVEL-2 PROBLEM STATEMENTS

Instructions:

1. Level-2 problem will be solved by 4 members team. The team is same as the project team of the Lab project.
2. Solution to be submitted to the respective lab instructor on the day of the lab project presentation.
3. Expected outcome of this exercise is working Verilog code, Test bench and output.
4. Team should demo the Verilog code to the respective lab instructor, and answer any questions asked related to the level-2 problem statement and Verilog syntax.
5. Lab instructor will assign Problem 1 for Half of the project team of respective section, Problem2 for the rest of the teams.

Problem Statement:

Problem1: Traffic Light Controller Built from K-map-Derived Flip-Flop Equations

Design a traffic light controller with 3 states, RED, GREEN, and YELLOW, cycling forever in that order, one clock tick per state. The requirement is to derive the actual next-state equations for the flip-flops directly from a K-map, and implement the FSM using those equations, rather than writing a behavioral case statement for the state transitions.

Interface:

- Inputs: clk, reset (synchronous, active high; goes to RED)
- Outputs: red, green, yellow (one-hot, only one high at a time)

State encoding: RED = 00, GREEN = 01, YELLOW = 10 (state bits S1 S0). The code 11 is unused.

Tasks:

1. Write the state transition truth table: current state S1 S0 to next state D1 D0, for RED to GREEN to YELLOW to RED. Treat the unused code 11 as a don't care.
2. Use a K-map to derive minimized Boolean equations for D1 and D0 directly from that truth table.
3. Derive the Moore output equations for red, green, and yellow directly from the state bits (each is a single minterm, no grouping needed).
4. Implement the FSM in Verilog using only the derived equations for D1, D0, red, green, and yellow, connected to a plain 2-bit register. Do not use a case statement anywhere in the design. Write a testbench that runs the FSM through at least 2 full RED-GREEN-YELLOW cycles and confirms the output sequence is correct.

Problem2: 4-Bit Counter FSM with Run/Hold Control and 7-Segment Output

Design a 4-bit counter controlled by a simple 2-state Moore FSM, with its value shown on a 7-segment display.

Interface:

- Inputs: clk, reset (synchronous, active high), run
- Outputs: count[3:0], seg[6:0] (common-cathode 7-segment pattern, active high)

Behavior: two states, RUN and HOLD. While run = 1, the FSM is in RUN and count increases by 1 every clock edge, wrapping from 1111 back to 0000. While run = 0, the FSM is in HOLD and count keeps its current value. seg is generated combinationally from count, showing the hexadecimal digit 0 through F.

Tasks:

1. Draw the 2-state state diagram for RUN and HOLD, labeling the transition condition on every arrow, including the self-loops.
2. Write the next-count logic as a truth table style description: in RUN, next count = count + 1 (mod 16); in HOLD, next count = count. Show how a 2-to-1 MUX selecting between (count + 1) and count, controlled by run, feeds the D inputs of four flip-flops.
3. Design the 7-segment decoder as a combinational module with a full truth table covering all 16 hex digits (0 through F), each mapped to its 7-segment pattern.
4. Implement the whole design in Verilog (the run-controlled counter plus the decoder) and write a testbench that checks: count increases while run = 1, count holds while run = 0, and count wraps from 1111 to 0000 correctly.